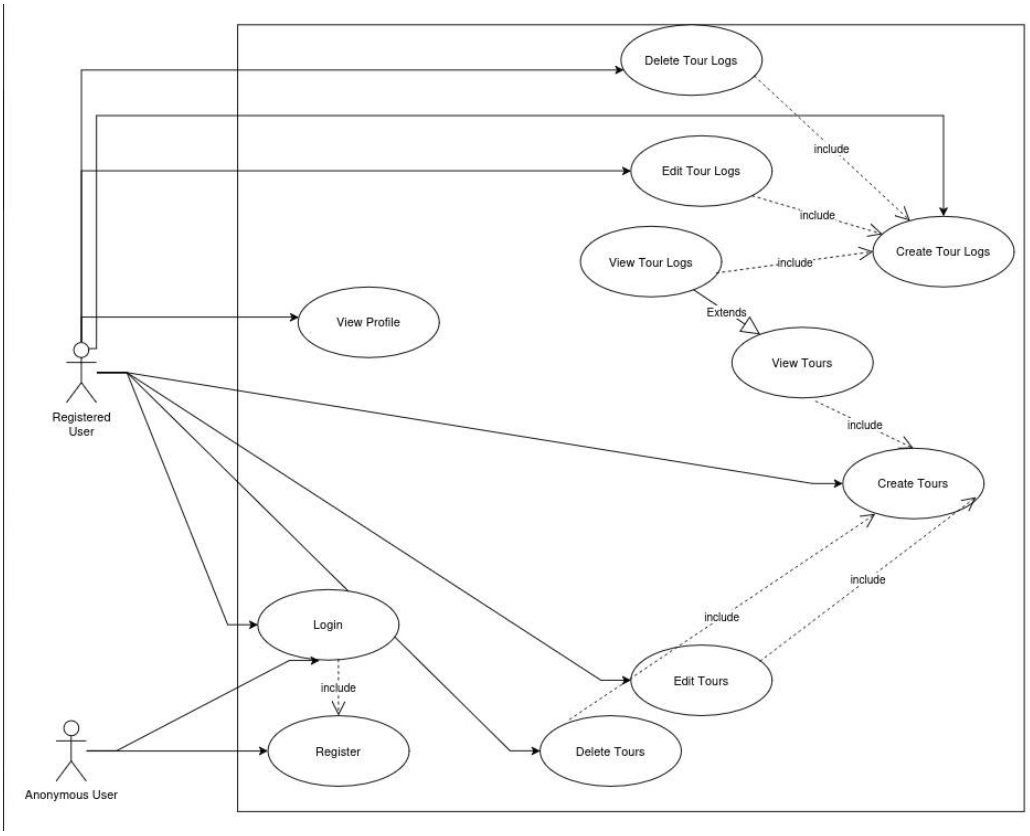
Tour Planner Protocol

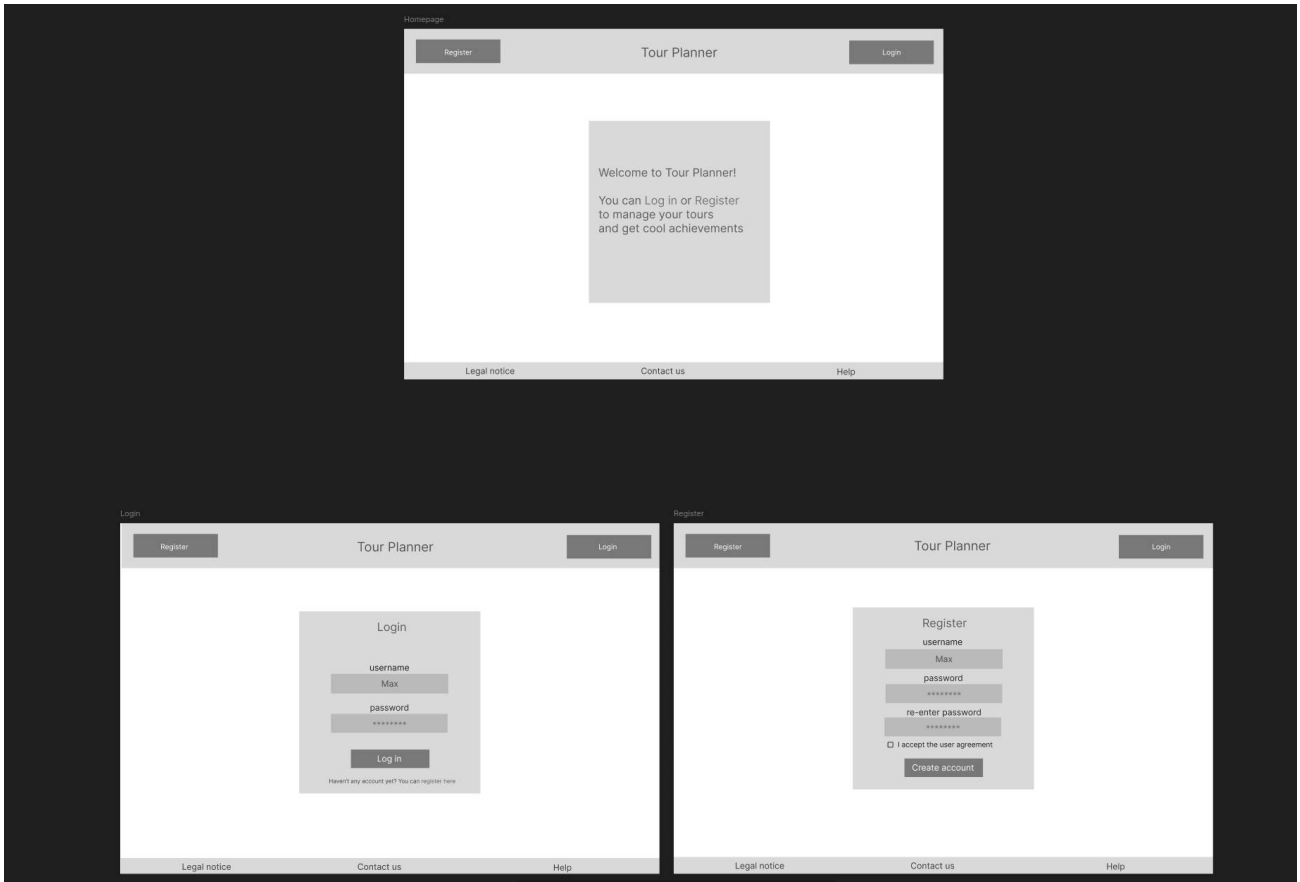
Olesja Nikel, David Nawloka

Git Link: https://github.com/lesjanikel/Tour_Planner.git

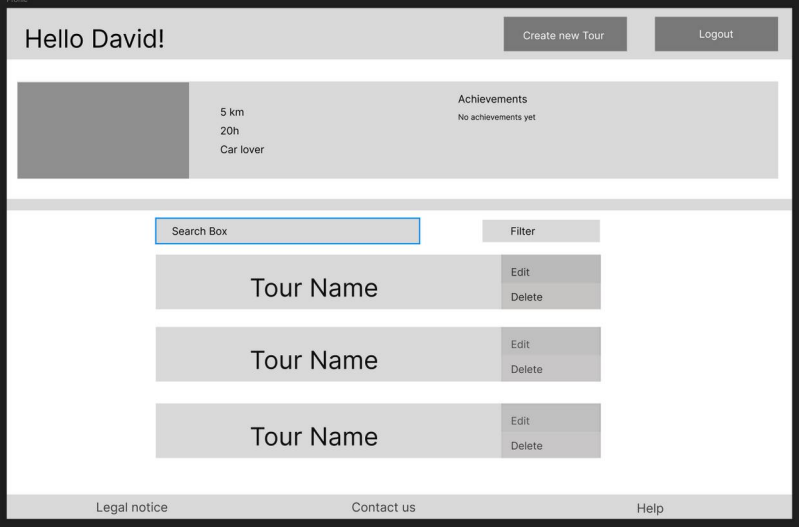
Usecase diagram:



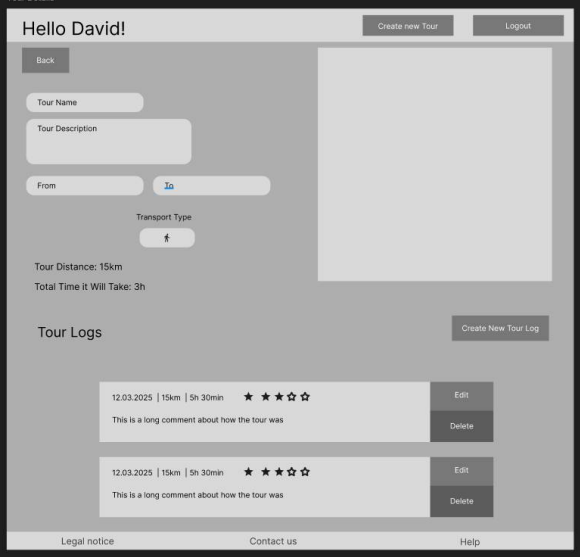
Tour Planner Wireframe:



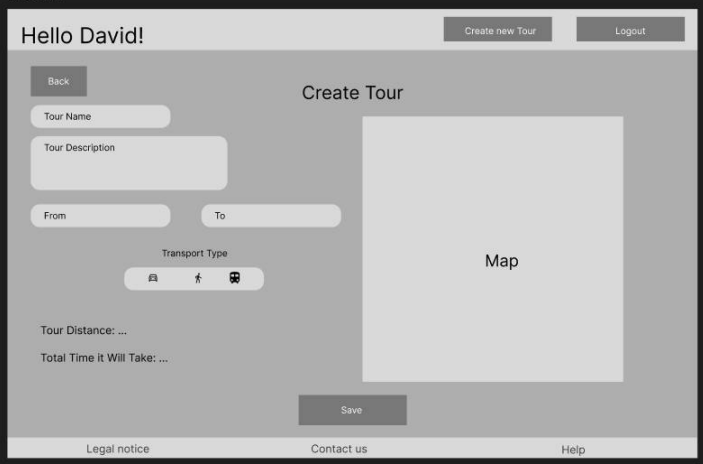
Profile



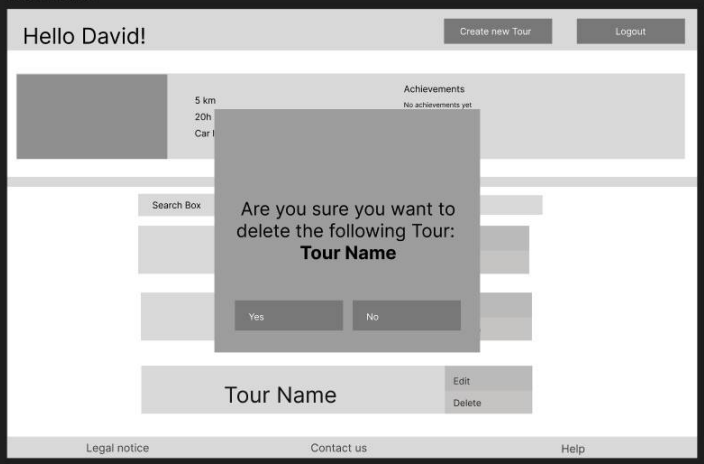
Tour Details



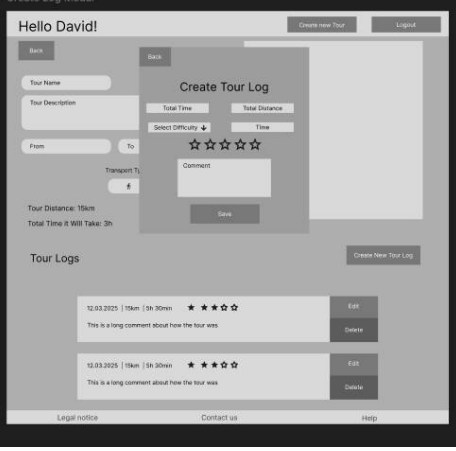
Create Tour



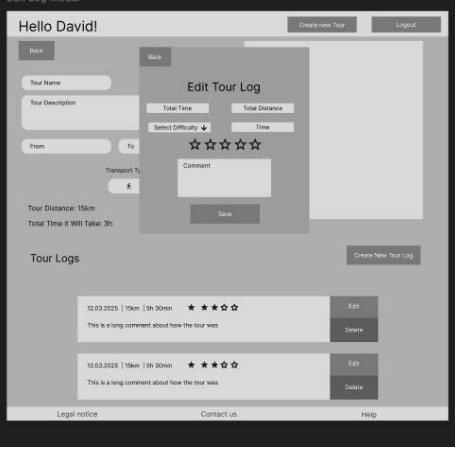
Delete Tour Modal



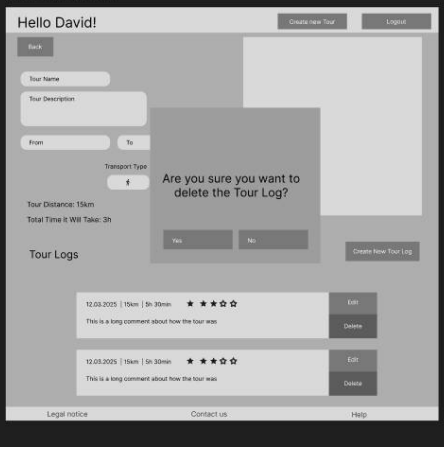
Create Log Modal



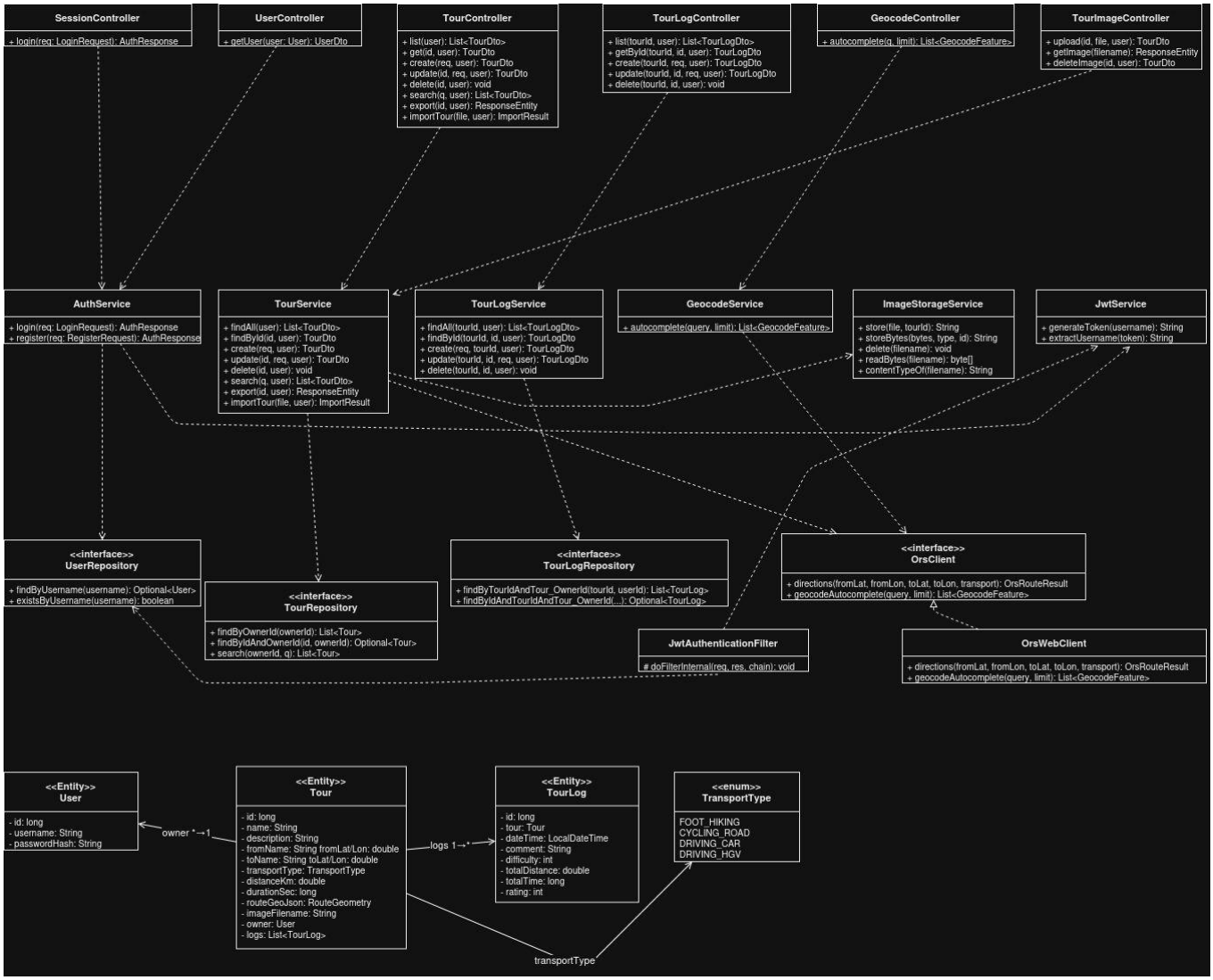
Edit Log Modal



Delete Tour Log Modal



Class Diagram



UX Description

We tried to keep the UX as simple and easy to understand as possible. To access most features one has to register and login. After that one has access to the tour dashboard, that displays some stats about the user (total distance/time in all tours, favorite transportation method) and some achievements the user has gotten. In addition to that, a logged in user can create a new tour by adding a tour name, description a start-/end location and a transport type. He then sees the tour in his dashboard. Clicking on the tour, takes the user to the tour detail view. There he can add tour logs - documenting his journey on the tour, including date, difficulty, distance, time and rating.

Library Decisions

Angular Signals: The tour dashboard needs to reactively display computed values like total distance, achievements, and filtered tour lists - all derived from a single tours array. Signals make these derivations straightforward with computed(), without the boilerplate of subscribing and unsubscribing that RxJS requires for simple state.

Spring Boot: The project requires a REST API with JWT security, JPA persistence, and validation. Spring Boot provides all of these through starters with minimal configuration, which suits the scope of a two-person project with a fixed deadline.

Spring Data JPA / Hibernate: Tour and TourLog entities have a clear relational structure (one-to-many). JPA handles the mapping automatically and allows JPQL for the full-text search query without writing raw SQL, which also eliminates SQL injection risk.

JWT (stateless auth): Tours and logs are strictly owner-scoped - every request needs to identify the user. JWT keeps the backend stateless (no session store needed) and works naturally with Angular's HTTP interceptor to attach the token to every request.

Leaflet: Lightweight and open-source map library that integrates cleanly with Angular. The route geometry from ORS is GeoJSON, which Leaflet renders directly - no conversion needed.

log4j2 via SLF4J: Spring Boot defaults to Logback, but the requirement specifies log4j. We added spring-boot-starter-log4j2 and excluded Logback. SLF4J acts as the logging facade so all code uses the same LoggerFactory API regardless of the underlying implementation.

Testcontainers (backend tests): Backend tests run against a real PostgreSQL container instead of an in-memory substitute, ensuring that JPA queries and constraints behave exactly as in production.

Base64 (image export): Encoding images as base64 strings in the export JSON was chosen to make tour exports fully self-contained - a single file includes all tour data and images without external references. This was a deliberate decision that simplifies import but increases file size.

Application Architecture

Layer Structure

The application follows a strict three-layer architecture:

- Presentation Layer: Angular 21 SPA with standalone components, Angular Signals for state management, and reactive forms.
- Business Layer (Spring Boot Services): TourService, TourLogService, AuthService, GeocodeService, ImageStorageService - all business rules enforced here.
- Data Access Layer (Spring Data JPA Repositories): TourRepository, TourLogRepository, UserRepository backed by PostgreSQL.

Backend Class Overview

Controllers (REST endpoints):

- TourController - CRUD for tours, search, import/export
- TourLogController - CRUD for tour logs
- TourImageController - image upload and retrieval
- GeocodeController - address autocomplete proxy
- SessionController - login and register
- UserController - profile information

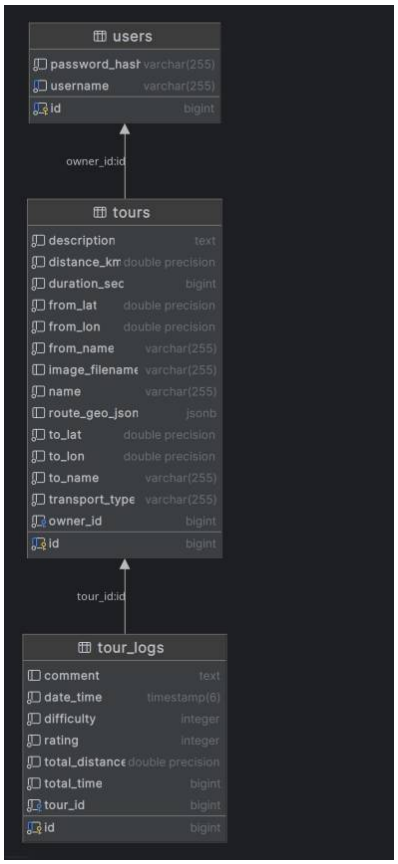
Models:

- User - id, username, passwordHash; owns tours
- Tour - id, name, description, from/to coordinates, transportType, distanceKm, durationSec, routeGeoJson, imageUrl, owner, logs
- TourLog - id, dateTime, comment, difficulty, totalDistance, totalTime, rating, tour

Security:

JwtService generates and validates HS256 JWT tokens. JwtAuthenticationFilter intercepts every request, extracts the token from the Authorization header and sets the Spring Security context.

Database Diagram



Frontend Structure

Services:

- TourService - holds tours signal, calls backend CRUD and search endpoints
- TourLogService - holds logs signal, calls backend log endpoints
- AuthService - holds user signal, stores JWT in localStorage
- GeocodeService - calls ORS geocode autocomplete
- MapFacadeService - facade over Leaflet to keep map concerns out of components
- ToastService - manages notification toasts via a signal
- AchievementsService - pure client-side logic computing achievement badges from tour data

Reusable component: AddressAutocomplete - a custom input component with debounced geocoding suggestions, emitting a structured GeocodeFeature on selection. Used in both TourForm (from/to fields).

Design Patterns Used

- Facade (MapFacadeService): hides Leaflet API complexity from components; components call initMap(), setRoute(), destroyMap() only.
- Repository (Spring Data JPA): TourRepository, TourLogRepository, UserRepository abstract all database access.
- Interceptor (Angular HTTP): AuthInterceptor attaches the JWT Bearer token to every outgoing request. ErrorInterceptor converts HTTP error responses to typed errors.
- MVVM (Angular): components expose signals as the ViewModel; templates bind directly to computed signals without additional logic.

Sequence Diagram: End-to-End Flow

The end-to-end sequence diagram is structured as a table with columns for each actor: User, Frontend, Backend, ORS, and DB. Arrow columns between actors indicate the direction of communication. A colored cell background marks the actor that is active in a given step. Steps where Backend communicates directly with DB use arrows through the ORS column area; the ORS cell remains empty and unfilled, indicating ORS does not participate in that step.

	User		Frontend		Backend		ORS		DB
PHASE 1 - REGISTER / LOGIN									
1	Fill register form + submit	->	POST /api/users (public endpoint)	->					
2					Check username BCrypt hash save user			->	save(user)
3				<-	generateToken() Return JWT				
4			Store JWT in localStorage navigate to /tours						
LOGIN (existing user)									
5	Fill login form + submit	->	POST /api/sessions (public endpoint)	->					
6					findByUsername()			->	user record
7				<-	verify BCrypt generateToken()				
8			Store JWT in localStorage navigate to /tours						
PHASE 2 - VIEW TOUR LIST									
9			GET /api/tours [JWT]	->					
10					JwtFilter validates JWT (all [JWT] requests)				
11					findByOwnerId()			->	tours
12			TourDto list update signal	<-					
13			Render list compute stats achievements						

PHASE 3 - CREATE TOUR										
14	Fill tour form + submit	->	Validate form (reactive)							
15	type address	->	GET /api/geocode/autocomplete ?text=...	->	geocodeAutocomplete()					
16	select location	<-	show dropdown	<-	return location list	<-	location suggestions			
17			POST /api/tours [JWT]	->						
18					orsClient .directions()					
19						<-	route geometry distance duration			
20					save(tour)			->	tour saved	
21			TourDto (201) navigate to tour detail	<-						
PHASE 4 - VIEW TOUR DETAIL + MAP										
22			GET /api/tours/:id [JWT]	->						
23					findByIdAndOwnerId()			->	tour	
24			TourDto received initMap() setRoute(GeoJSON)	<-						
25			GET /api/tours/:id/logs [JWT]	->						
26					findByTourId()			->	logs	
27			logs received render log list	<-						
PHASE 5 - UPLOAD IMAGE										
28			POST /api/tours/:id/image [JWT, multipart]	->						
29					Store file update imageFilename			->	updated	
30			updated TourDto update tour signal	<-						
PHASE 6 - ADD TOUR LOG										
31	Fill log form + submit	->	POST /api/tours/:id/logs [JWT]	->						
32					save(log)			->	log saved	
33			TourLogDto (201) update signal navigate to tour detail	<-						
PHASE 7 - UPDATED STATS + ACHIEVEMENTS										
34	Navigate to tour list	->	GET /api/tours [JWT]	->						
35					findByOwnerId()			->	tours	
36			tours received recompute stats achievements	<-						
PHASE 8 - EXPORT TOUR										
37			GET /api/tours/:id/export [JWT]	->						
38					Load tour, logs base64 image serialize JSON			->	fetch all	
39	Download JSON file	<-	JSON file	<-						
PHASE 9 - IMPORT TOUR										
40			POST /api/tours/import [JWT, multipart]	->						
41					Parse JSON check duplicates save			->	save tours	
42			ImportResult (imported, duplicates) update signal show toast	<-						

End-to-End Tests

The end-to-end test suite uses Spring Boot full application context with a real PostgreSQL instance provided by Testcontainers, exercising the complete HTTP stack without mocking any application layer. Only the external OpenRouteService client is replaced with a test double to avoid network dependency.

- protectedEndpoint_requiresAuthentication: Verifies that any request to a protected endpoint without a valid JWT is rejected with HTTP 401, confirming that the security filter chain is active and correctly positioned in the request pipeline.
- fullFlow_registerLoginCreateSearch: Exercises the complete user journey - registration, login, tour creation, and full-text search - across real HTTP requests against a real database. This test confirms that all layers integrate correctly end-to-end and that the JWT issued during login is accepted by subsequent authenticated requests.

Unit Tests

Tests were written for classes with non-trivial logic where a mistake would be silent or hard to detect at runtime. Static pages and pure HTTP-delegation classes (no branching logic) were excluded.
Frontend tests were added as a learning exercise to gain experience with Angular testing.

Backend

- JwtServiceTest: The JwtService is responsible for generating and verifying authentication tokens, which are central to the security of the entire application. A unit test confirms that a generated token correctly embeds the username as its subject, verifying the fundamental token generation behaviour.
- AuthServiceTest: The AuthService enforces the registration and login rules that control access to the system. Unit tests verify that duplicate usernames are rejected before any persistence occurs and that valid credentials produce a correctly issued token, protecting against unauthorized account creation and access. Correct error throwing is also tested.
- TourServiceTest / TourLogServiceTest: Both services enforce owner-scoped data access by querying only resources that belong to the authenticated user. Unit tests verify that requests for non-existent or unowned resources throw the expected exception, ensuring that the access boundary between users cannot be silently bypassed.
- GeocodeServiceTest: The GeocodeService delegates queries to the external OpenRouteService API, which has usage limits. Unit tests confirm that blank or empty input does not trigger an API call, preventing unnecessary quota consumption and potential runtime errors from the external service.
- GlobalExceptionHandlerTest: The GlobalExceptionHandler translates application exceptions into HTTP responses consumed by the frontend. Unit tests verify that the most critical mappings are correct: NotFoundException produces a 404 response and BadCredentialsException produces a 401 response, ensuring that the frontend receives consistent and meaningful error information.
- ImageStorageServiceTest: The ImageStorageService performs file system operations on user-uploaded content with content-type validation. Unit tests confirm that unsupported file types are rejected before any file system interaction occurs and that valid uploads are stored correctly, preventing storage corruption.

Frontend

- AchievementsService: The AchievementsService contains pure business logic with no external dependencies, making it well-suited for isolated unit testing. The correctness of achievement evaluation relies on correctly aggregating data across all tours, which makes automated verification of the accumulation logic essential.
- AuthService / Guards: The AuthService manages the authenticated user session, and the route guards rely on its state to protect restricted routes. Unit tests verify that the session correctly reflects authentication status and that route guards block access for unauthenticated users.
- TourList computed signals: The filteredTours and totalDistance values are computed signals derived from the tour data and active filters. A regression in this logic would cause incorrect data to be displayed without any visible error, making automated verification of the derivation logic important.
- Form validation (TourForm, TourLogForm, Register, Login): Input validation in the tour and log forms as well as in the registration and login flows prevents malformed data from being submitted to the backend. Unit tests confirm that the defined validation constraints are correctly enforced by the form controls.

- TourLogService (HTTP): The TourLogService maintains an in-memory signal that must remain consistent with the server state after each operation. Unit tests using HttpTestingController verify that delete and update operations correctly synchronise the local signal with the server response.
- ToastService / AddressAutocomplete: ToastService and AddressAutocomplete are shared components used across multiple features of the application. Unit tests ensure their core behaviour remains stable, as any regression would have a broad impact on the user experience.

Unique Feature: Achievements System

The Achievements system is a purely client-side gamification feature implemented in AchievementsService. It analyzes all of the user's tours and awards badges based on five dimensions:

- Tour count: First Tour Completed / Experienced Traveller / Tour Master (thresholds: 1 / 5 / 10 tours).
- Total distance: Long Distance Traveller / 100 km Completed / Road Legend / World Explorer (thresholds: 50 / 100 / 200 / 500 km across all tours).
- Total time: Time Explorer / Time Master / Travel Veteran (thresholds: 300 / 600 / 1200 minutes).
- Longest single tour: Marathon Tour / Extreme Route (thresholds: 25 / 50 km).
- Transport variety: Multi-Transport Traveller / Transport Master (thresholds: 2 / 4 distinct transport types used).
- Badges are displayed in the tour dashboard header. Because the logic is pure (no side effects, no HTTP calls), it is fully unit-tested in isolation.

Lessons Learned

- Bidirectional JPA relationships (@OneToMany/@ManyToOne) require explicit handling of JSON serialization to prevent infinite recursion - this needs to be considered when designing the entity model.
- It is good to have the hard application startup crash in order to prevent the silent crash during application usage.
- Validation constraints should be applied at both the application and database layers. @NotBlank catches invalid input early with a descriptive error response; @Column(nullable = false) enforces the same constraint at the schema level as a safety net. Relying on only one layer leaves a gap - missing @Valid on a controller method silently allows null values through to the database.
- Logging infrastructure is most valuable when integrated during development, as it directly supports debugging and reduces investigation time.
- The image can be converted to a Base64 string to be exported if the image size limits are set up.

Time Tracking

- Use case diagram, wireframes / UI prototype: ~6h
- Architecture planning: ~2h
- Use case diagram, wireframes and UI prototype: ~3h
- Project setup, Docker, database schema, Spring Boot skeleton: ~7h
- Authentication (JWT, register, login, guards): ~6h
- Logging setup (SLF4J / log4j, adding log statements throughout the application): ~2h
- Tour CRUD backend + frontend: ~8h
- Tour Log CRUD backend + frontend: ~4h
- OpenRouteService integration (routing + geocoding): ~4h
- Leaflet map integration (MapFacadeService): ~2h
- Search, computed attributes (popularity, child-friendliness): ~4h
- Import / Export feature: ~4h
- Image upload feature: ~3h
- Achievements system (unique feature): ~2h
- Logging (log4j/SLF4J setup + adding log statements): ~2h
- Unit tests (frontend + backend): ~4h
- Styling, UX polish, bug fixes: ~5h
- Documentation: ~2h
- Total: ~65h